3638 lines (3638 loc) · 80.4 KB

Experiment 1

NAME :- K.S. RITHVIK

SAP_ID :- 590030656

COURSE :- MCA

Question 1 📝 Aim

To install Python and understand the difference between Interactive Mode and Scripting Mode.

Explanation

Python can be executed in two ways: Interactive Mode and Scripting Mode. Interactive Mode executes one command at a time, while Scripting Mode runs an entire Python program saved in a .py file. This experiment also checks whether Python has been installed successfully.

In [2]:
```
import sys

print("Python is installed successfully.")
print("Python Version:", sys.version)
```

```
Python is installed successfully.
Python Version: 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang
21.0.0 (clang-2100.1.1.101)]
```

Question 2 📝 Aim

Write a Python program to print different strings.

Explanation

The print() function displays text on the screen. The \n escape sequence moves the cursor to the next line, allowing text to be printed on separate lines.

In [3]:
```
print("Hello Everyone !!!")

print("Hello")
print("World")

print("Hello\nWorld")

print("KS RITHVIK's date of birth is 23/02/2006")
```

```
Hello Everyone !!!
Hello
World
Hello
World
```

```
KS RITHVIK's date of birth is 23/02/2006
```

Question 3 📝 Aim

Create a variable and print its value.

Explanation

A variable is used to store data in memory. Here, the variable x stores the string "Hello" and print() displays it.

In [4]:
```
x = "Hello"

print(x)
```

```
Hello
```

Question 4 📝 Aim

Print different data types in Python.

Explanation

Python supports multiple data types such as integers, floating-point numbers, strings, booleans, and lists. This program demonstrates each type by storing values in variables and printing them.

In [5]:
```
integer_value = 25
float_value = 3.14
string_value = "Python Programming"
boolean_value = True
list_value = [10, 20, 30]

print("Integer:", integer_value)
print("Float:", float_value)
print("String:", string_value)
print("Boolean:", boolean_value)
print("List:", list_value)
```

```
Integer: 25
Float: 3.14
String: Python Programming
Boolean: True
List: [10, 20, 30]
```

Question 5 📝 Aim

Concatenate first name and last name.

Explanation

String concatenation means joining two or more strings together. Here, the first name and last name are combined with a space to form the full name

In [6]:
```
first_name = "KS"
last_name = "RITHVIK"
```

```
full_name = first_name + " " + last_name

print(full_name)
```

```
KS RITHVIK
```

Question 6 📝 Aim

Print a name in the format FirstName (Nickname) LastName.

Explanation

Python's f-string formatting makes it easy to insert variables into a string. This improves readability and makes the code simpler than using multiple concatenations.

In [7]:
```
first_name = "KS"
nickname = "Rithvik"
last_name = "R"

print(f"{first_name} ({nickname}) {last_name}")
```

```
KS (Rithvik) R
```

Question 7 📝 Aim

Store and display student details.

Explanation

Variables are used to store personal information such as name, SAP ID, date of birth, programme, and semester. The stored values are then displayed using the print() function.

In [8]:
```
name = "KS RITHVIK"
sap_id = "590030656"
dob = "23/02/2006"
programme = "MCA (AI & ML)"
semester = 1

print("Name:", name)
print("SAP ID:", sap_id)
print("Date of Birth:", dob)
print("Programme:", programme)
print("Semester:", semester)
```

```
Name: KS RITHVIK
SAP ID: 590030656
Date of Birth: 23/02/2006
Programme: MCA (AI & ML)
Semester: 1
```

Experiment 2

Experiment 2: Input Statements and Operators

Question 1 Aim

To perform basic arithmetic operations on two integer numbers.

Explanation

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication, and division. In this program, two numbers are stored in variables and different arithmetic operations are performed.

In [ ]:
```
x = 9
y = 7

print("Addition:", x + y)
print("Subtraction:", x - y)
print("Multiplication:", x * y)
print("Division:", x / y)
```

```
Addition: 16
Subtraction: 2
Multiplication: 63
Division: 1.2857142857142858
```

Question 2 Aim

To calculate the area of a circle using the radius entered by the user.

Explanation

The area of a circle is calculated using the formula π × r². The program accepts the radius as input from the user and uses the math.pi constant for an accurate value of π.

In [2]:
```
import math

radius = float(input("Enter the radius: "))

area = math.pi * radius ** 2

print("Area of the circle =", area)
```

```
Area of the circle = 78.53981633974483
```

Question 3 Aim

To evaluate the expression (x + y)².

Explanation

The program calculates the square of the sum of two numbers using arithmetic operators. First, the values are added together, and then the result is multiplied by itself.

In [3]:
```
x = 4
y = 3

result = (x + y) * (x + y)

print("Result =", result)
```

```
Result = 49
```

Question 4 📝 Aim

To display the given values of two variables.

Explanation

Variables are used to store values in memory. This program assigns values to two variables and prints them using the print() function.

In [4]:
```
x = 4
y = 3

print("Value of x =", x)
print("Value of y =", y)
```

```
Value of x = 4
Value of y = 3
```

Question 5 📝 Aim

To display the expected and actual result of the expression $(x + y)^2$.

Explanation

This program verifies the calculation by displaying both the expected result and the actual result computed by Python. It helps in checking whether the program produces the correct output.

In [5]:
```
x = 4
y = 3

print("Expected Output:", 49)
print("Actual Output:", (x + y) * (x + y))
```

```
Expected Output: 49
Actual Output: 49
```

Experiment 3

Experiment 3: Conditional Statements

Question 1 📝 Aim

To check whether a given number is divisible by both 3 and 5.

Explanation

This program uses the if statement along with the logical and operator. A number is considered divisible by both 3 and 5 only if both conditions are true.

In [1]:
```
num = 45

if num % 3 == 0 and num % 5 == 0:
    print(num, "is divisible by both 3 and 5")
else:
    print(num, "is not divisible by both 3 and 5")
```

```
45 is divisible by both 3 and 5
```

Question 2 Aim

To check whether a given number is a multiple of 5.

Explanation

The modulus (%) operator returns the remainder after division. If the remainder is 0, then the number is a multiple of 5.

In [2]:
```
num = 37

if num % 5 == 0:
    print(num, "is a multiple of 5")
else:
    print(num, "is not a multiple of 5")
```

```
37 is not a multiple of 5
```

Question 3 Aim

To find the greatest among two numbers.

Explanation

The program compares two numbers using if, elif, and else. It displays the larger number, and if both numbers are the same, it prints that the numbers are equal.

In [3]:
```
a = 15
b = 15

if a > b:
    print("Greatest number is", a)
elif b > a:
    print("Greatest number is", b)
else:
    print("Numbers are equal")
```

```
Numbers are equal
```

Question 4 Aim

To find the greatest among three numbers.

Explanation

The program compares three numbers using multiple if-elif-else conditions. It checks each number against the other two and prints the largest value

In [4]:

In [4]:
```
a = 12
b = 45
c = 32

if a > b and a > c:
    print("Greatest number is", a)
elif b > a and b > c:
    print("Greatest number is", b)
else:
    print("Greatest number is", c)
```

```
Greatest number is 45
```

Question 5 Aim

To determine whether the roots of a quadratic equation are real or imaginary and display them.

Explanation

The discriminant (b² - 4ac) determines the nature of the roots. If it is positive or zero, the roots are real; otherwise, they are imaginary. The cmath module is used because it can handle both real and complex roots.

In [5]:
```
import cmath

a = 1
b = 2
c = 5

discriminant = b**2 - 4*a*c

root1 = (-b + cmath.sqrt(discriminant)) / (2*a)
root2 = (-b - cmath.sqrt(discriminant)) / (2*a)

if discriminant >= 0:
    print("Roots are real")
else:
    print("Roots are imaginary")

print("Root 1:", root1)
print("Root 2:", root2)
```

```
Roots are imaginary
Root 1: (-1+2j)
Root 2: (-1-2j)
```

Question 6 Aim

To check whether a given year is a leap year.

Explanation

A leap year is divisible by 400, or divisible by 4 but not by 100. The program checks these conditions and prints whether the year is a leap year.

In [6]:
```
year = 2024

if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
```

```
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

```
2024 is a leap year
```

Question 7 📝 Aim

To display the next calendar date.

Explanation

The datetime module is used to work with dates. By adding one day using timedelta(days=1), the program automatically calculates the next calendar date.

In [7]:

```
from datetime import date, timedelta

d = date(2005, 9, 20)

next_day = d + timedelta(days=1)

print("Input Date:", d.strftime("%d-%m-%Y"))
print("Next Date :", next_day.strftime("%d-%m-%Y"))
```

```
Input Date: 20-09-2005
Next Date : 21-09-2005
```

Question 8 📝 Aim

To prepare a student's grade sheet based on marks obtained in five subjects.

Explanation

The program calculates the total marks, percentage, and CGPA. It then uses conditional statements to assign a grade according to the CGPA range, making the grading process automatic and accurate.

In [9]:

```
def get_grade(cgpa):
    if 0.0 <= cgpa <= 3.4:
        return "F"
    if 3.5 <= cgpa <= 5.0:
        return "C+"
    if 5.1 <= cgpa <= 6.0:
        return "B"
    if 6.1 <= cgpa <= 7.0:
        return "B+"
    if 7.1 <= cgpa <= 8.0:
        return "A"
    if 8.1 <= cgpa <= 9.0:
        return "A+"
    if 9.1 <= cgpa <= 10.0:
        return "O (Outstanding)"
    return "Invalid CGPA"

name = "Rohit Sharma"
roll_no = "R17234512"
sap_id = "50005673"
semester = 1
course = "B.Tech. CSE AI & ML"
```

```
marks = {
    "PDS": 70,
    "Python": 80,
    "Chemistry": 90,
    "English": 60,
    "Physics": 65,
}

total = sum(marks.values())
percentage = (total / (len(marks) * 100)) * 100
cgpa = round(percentage / 10, 1)
grade = get_grade(cgpa)

print("Sample Gradesheet")
print("Name:", name)
print("Roll Number:", roll_no, "SAPID:", sap_id)
print("Sem:", semester, "Course:", course)
print("Subject name: Marks")
for subject, score in marks.items():
    print(f"{subject}: {score}")
print(f"Percentage: {percentage:.2f}%")
print("CGPA:", cgpa)
print("Grade:", grade)
```

```
Sample Gradesheet
Name: Rohit Sharma
Roll Number: R17234512 SAPID: 50005673
Sem: 1 Course: B.Tech. CSE AI & ML
Subject name: Marks
PDS: 70
Python: 80
Chemistry: 90
English: 60
Physics: 65
Percentage: 73.00%
CGPA: 7.3
Grade: A
```

Experiment 4

Experiment 4: Loops

Question 1 📝 Aim

To find the factorial of a given number using a loop.

Explanation

A factorial is the product of all positive integers from 1 to the given number. The for loop is used to multiply each number one by one and calculate the factorial.

In [1]:

```
num = int(input("Enter a number: "))

factorial = 1

for i in range(1, num + 1):
    factorial *= i
```

```
print("Factorial =", factorial)
```

```
Factorial = 120
```

Question 2 📝 Aim

To check whether a given number is an Armstrong number.

Explanation

An Armstrong number is a number whose sum of each digit raised to the power of the number of digits is equal to the original number. The program separates each digit using a loop and verifies the condition.

In [2]:
```
num = int(input("Enter a number: "))

order = len(str(num))
temp = num
sum = 0

while temp > 0:
    digit = temp % 10
    sum += digit ** order
    temp //= 10

if sum == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")
```

```
153 is an Armstrong number
```

Question 3 📝 Aim

To print the Fibonacci series up to a given number of terms.

Explanation

The Fibonacci series starts with 0 and 1. Each new number is the sum of the previous two numbers. A loop is used to generate and display the required number of terms.

In [3]:
```
terms = int(input("Enter number of terms: "))

a = 0
b = 1

for i in range(terms):
    print(a, end=" ")
    a, b = b, a + b
```

```
0 1 1 2 3 5 8 13
```

Question 4 📝 Aim

To check whether a given number is prime.

Explanation

A prime number has only two factors: 1 and itself. The program checks divisibility from 2 up to the square root of the number using a loop. If any divisor is found, the number is not prime.

In [4]:
```
num = int(input("Enter a number: "))

if num < 2:
    print("Not a Prime Number")
else:
    prime = True

    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            prime = False
            break

    if prime:
        print("Prime Number")
    else:
        print("Not a Prime Number")
```

```
Prime Number
```

Question 5 Aim

To check whether a number is a palindrome.

Explanation

A palindrome number remains the same when its digits are reversed. The program reverses the number using a loop and compares it with the original number.

In [1]:
```
num = int(input("Enter a number: "))

original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num //= 10

if original == reverse:
    print("Palindrome Number")
else:
    print("Not a Palindrome Number")
```

```
Palindrome Number
```

Question 6 Aim

To find the sum of digits of a number.

Explanation

The program extracts each digit using the modulus operator and adds it to a running total. The loop continues until all digits have been processed.

In [2]:
```
num = int(input("Enter a number: "))

sum_digits = 0

while num > 0:
    sum_digits += num % 10
    num //= 10

print("Sum of digits =", sum_digits)
```

```
Sum of digits = 15
```

Question 7 Aim

To count and print all numbers divisible by 5 or 7 between 1 and 100.

Explanation

The loop checks every number from 1 to 100. If a number is divisible by either 5 or 7, it is printed and counted.

In [3]:
```
count = 0

for i in range(1, 101):
    if i % 5 == 0 or i % 7 == 0:
        print(i, end=" ")
        count += 1

print("\nTotal Count =", count)
```

```
5 7 10 14 15 20 21 25 28 30 35 40 42 45 49 50 55 56 60 63 65 70 75 77 80 84
85 90 91 95 98 100
Total Count = 32
```

Question 8 Aim

To convert all lowercase letters to uppercase.

Explanation

The program uses a loop to read each character in the string. The upper() method converts lowercase letters into uppercase while leaving other characters unchanged

In [4]:
```
text = input("Enter a string: ")

result = ""

for ch in text:
    result += ch.upper()

print("Uppercase String:", result)
```

```
Uppercase String: HELLO WORLD
```

Question 9 Aim

To print all prime numbers between 1 and 100.

Explanation

The program checks every number from 2 to 100. For each number, a nested loop verifies whether it has any divisor other than 1 and itself. Only prime numbers are displayed.

In [5]:
```
for num in range(2, 101):

    prime = True

    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            prime = False
            break

    if prime:
        print(num, end=" ")
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

Question 10 Aim

To print the multiplication table of a given number.

Explanation

The program uses a for loop to multiply the given number by values from 1 to 10. Each result is displayed in a standard multiplication table format.

In [6]:
```
num = int(input("Enter a number: "))

for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

```
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80
```

Experiment 5

Experiment 5: Strings and Sets

Question 1 Aim

To count the number of capital letters in a given string.

Explanation

The program checks each character in the string using the isupper() method. If the

The program checks each character in the string using the isupper() method. If the character is an uppercase letter, the counter is increased by one. Finally, the total number of capital letters is displayed.

In [1]:
```
text = "PyThOn LAB 2026"

count_caps = sum(1 for ch in text if ch.isupper())

print("Capital letters:", count_caps)
```

```
Capital letters: 6
```

Question 2 📝 Aim

To count the total number of vowels in a given string.

Explanation

The program compares each character with the vowels (a, e, i, o, u) in both uppercase and lowercase. Whenever a vowel is found, the count is increased and displayed at the end.

In [2]:
```
text = "Welcome to Python Programming"

vowels = "aeiouAEIOU"

count_vowels = sum(1 for ch in text if ch in vowels)

print("Number of vowels:", count_vowels)
```

```
Number of vowels: 8
```

Question 3 📝 Aim

To print each word of a sentence on a separate line.

Explanation

The split() method divides the sentence into individual words. A for loop then prints each word on a new line.

In [3]:
```
sentence = "Python is easy to learn"

for word in sentence.split():
    print(word)
```

```
Python
is
easy
to
learn
```

Question 4 📝 Aim

To count the number of occurrences of a substring in a given string.

Explanation

The program searches for the substring using the find() method. Each time the substring is found, the count is increased and the search continues from the next position until no more occurrences are found.

In [4]:
```
main_string = "ABCDCDC"
sub_string = "CDC"

count = 0
start = 0

while True:
    index = main_string.find(sub_string, start)

    if index == -1:
        break

    count += 1
    start = index + 1

print("Occurrences:", count)
```

```
Occurrences: 2
```

Question 5 Aim

To count the occurrence of each alphabet in a string (case-insensitive).

Explanation

The program reads every alphabet in the string and converts it to uppercase. A dictionary stores the frequency of each letter so that uppercase and lowercase letters are counted together.

In [5]:
```
text = "ABaBCbGc"

frequency = {}

for ch in text:
    if ch.isalpha():
        key = ch.upper()
        frequency[key] = frequency.get(key, 0) + 1

print(frequency)
```

```
{'A': 2, 'B': 3, 'C': 2, 'G': 1}
```

Question 6 Aim

To count the number of unique words in a sentence using sets.

Explanation

The sentence is first converted into lowercase and split into words. A set is then created because sets automatically remove duplicate values. The total number of unique words is obtained using len().

In [6]:
```
sentence = "python is simple and python is powerful"
```

```
sentence = "python is simple and python is powerful"

words = sentence.lower().split()

unique_words = set(words)

print("Unique words:", unique_words)
print("Count:", len(unique_words))
```

```
Unique words: {'and', 'powerful', 'simple', 'python', 'is'}
Count: 5
```

Question 7 📝 Aim

To perform basic set operations on two sets of fruits.

Explanation

The program uses set operators to find common fruits, fruits present only in the first set, and all unique fruits from both sets. These operations demonstrate the usefulness of sets in handling collections of unique elements.

In [7]:

```
s1 = {"apple", "banana", "mango", "grapes"}
s2 = {"banana", "orange", "kiwi", "mango"}

print("Fruits in both sets:", s1 & s2)
print("Fruits only in s1:", s1 - s2)
print("Total unique fruits:", len(s1 | s2))
```

```
Fruits in both sets: {'mango', 'banana'}
Fruits only in s1: {'apple', 'grapes'}
Total unique fruits: 6
```

Experiment 6

Experiment 6: Lists, Tuples and Dictionaries

Question 1 📝 Aim

To scan n values in the range 0–3 and count the number of occurrences of each value.

Explanation

A list is used to store the entered values. Another list is used to maintain the count of each number from 0 to 3. Every time a number is entered, its corresponding count is increased.

In [2]:

```
n = int(input("Enter the number of values: "))

values = []
count = [0, 0, 0, 0]

for i in range(n):
    num = int(input("Enter a value (0-3): "))
```

```
    if 0 <= num <= 3:
        values.append(num)
        count[num] += 1
    else:
        print("Invalid Input")

print("\nOccurrences:")
for i in range(4):
    print(f"{i} occurred {count[i]} time(s)")
```

```
Occurrences:
0 occurred 1 time(s)
1 occurred 2 time(s)
2 occurred 2 time(s)
3 occurred 1 time(s)
```

Question 2 📝 Aim

To create a tuple of numeric values and find its average.

Explanation

A tuple stores multiple values that cannot be modified after creation. The sum() function calculates the total of all elements, and the average is obtained by dividing the total by the number of elements.

In [3]:

```
numbers = (12, 25, 18, 30, 15)

average = sum(numbers) / len(numbers)

print("Tuple:", numbers)
print("Average =", average)
```

```
Tuple: (12, 25, 18, 30, 15)
Average = 20.0
```

Question 3 📝 Aim

To input the scores of students and print the runner-up score.

Explanation

The scores are stored in a list. Duplicate scores are removed using a set, and the remaining values are sorted. The second highest score is displayed as the runner-up.

In [4]:

```
scores = list(map(int, input("Enter student scores: ").split()))

unique_scores = sorted(set(scores))

print("Runner-up Score =", unique_scores[-2])
```

```
Runner-up Score = 40
```

Question 4 📝 Aim

To create a dictionary containing names and cities, and perform different dictionary operations.

Explanation

A dictionary stores data as key-value pairs. The program displays all names, city names, each person's details, and the total number of entries stored in the dictionary.

In [5]:
```
students = {
    "Rithvik": "Hyderabad",
    "Rahul": "Delhi",
    "Priya": "Mumbai",
    "Sneha": "Chennai"
}

print("Names:")
for name in students.keys():
    print(name)

print("\nCities:")
for city in students.values():
    print(city)

print("\nStudent Details:")
for name, city in students.items():
    print(name, "->", city)

print("\nTotal Students =", len(students))
```

```
Names:
Rithvik
Rahul
Priya
Sneha

Cities:
Hyderabad
Delhi
Mumbai
Chennai

Student Details:
Rithvik -> Hyderabad
Rahul -> Delhi
Priya -> Mumbai
Sneha -> Chennai

Total Students = 4
```

Question 5 📝 Aim

To store movie details in a dictionary and perform different search operations.

Explanation

Each movie is stored with its release year, profit status, and director. The program displays all movies, filters movies released before 2015, identifies profitable movies, and searches for movies by a specific director.

In [6]:
```
movies = {
    "3 Idiots": {"Year": 2009, "Profit": True, "Director": "Rajkumar Hirani
    "KGF": {"Year": 2018, "Profit": True, "Director": "Prashanth Neel"},
    "Ra.One": {"Year": 2011, "Profit": False, "Director": "Anubhav Sinha"},
    "Dangal": {"Year": 2016, "Profit": True, "Director": "Nitesh Tiwari"}
```

```
    "Dangal": {"Year": 2016, "Profit": True, "Director": "Nitesh Tiwari"}
}

print("All Movie Details:")
for movie, details in movies.items():
    print(movie, ":", details)

print("\nMovies Released Before 2015:")
for movie, details in movies.items():
    if details["Year"] < 2015:
        print(movie)

print("\nProfitable Movies:")
for movie, details in movies.items():
    if details["Profit"]:
        print(movie)

director = input("\nEnter Director Name: ")

print("\nMovies by", director)
for movie, details in movies.items():
    if details["Director"].lower() == director.lower():
        print(movie)
```

```
All Movie Details:
3 Idiots : {'Year': 2009, 'Profit': True, 'Director': 'Rajkumar Hirani'}
KGF : {'Year': 2018, 'Profit': True, 'Director': 'Prashanth Neel'}
Ra.One : {'Year': 2011, 'Profit': False, 'Director': 'Anubhav Sinha'}
Dangal : {'Year': 2016, 'Profit': True, 'Director': 'Nitesh Tiwari'}

Movies Released Before 2015:
3 Idiots
Ra.One

Profitable Movies:
3 Idiots
KGF
Dangal

Movies by
```

Experiment 7

Experiment 7: Functions

Question 1 📝 Aim

To write a function that finds the maximum and minimum values from a sequence without using the built-in max() and min() functions.

Explanation

The function starts by assuming the first element is both the maximum and minimum. It then compares every remaining element in the sequence and updates the maximum or minimum whenever needed.

In [1]:

```
def find_max_min(sequence):
```

```
    maximum = sequence[0]
    minimum = sequence[0]

    for value in sequence[1:]:
        if value > maximum:
            maximum = value
        if value < minimum:
            minimum = value

    return maximum, minimum


numbers = [12, 45, 3, 89, 25]

maximum, minimum = find_max_min(numbers)

print("Maximum =", maximum)
print("Minimum =", minimum)
```

```
Maximum = 89
Minimum = 3
```

Question 2 📝 Aim

To write a function that returns the sum of cubes of all positive integers less than a given number.

Explanation

The function uses a for loop to calculate the cube of every number from 1 to n-1. Each cube is added to a running total, which is returned at the end.

In [2]:

```
def sum_of_cubes_below(n):
    total = 0

    for i in range(1, n):
        total += i ** 3

    return total


n = 6

print("Sum of cubes =", sum_of_cubes_below(n))
```

```
Sum of cubes = 225
```

Question 3 📝 Aim

To print numbers from 1 to n using recursion.

Explanation

Recursion is a technique in which a function calls itself. The function first reaches the base case and then prints the numbers while returning from each recursive call.

In [3]:

```
def print_1_to_n(n):

    if n == 0:
        return
```

```
    print_1_to_n(n - 1)

    print(n, end=" ")


print_1_to_n(10)
```

```
1 2 3 4 5 6 7 8 9 10
```

Question 4 📝 Aim

To print the Fibonacci series using recursion.

Explanation

The Fibonacci sequence is generated by adding the previous two numbers. A recursive function calculates each Fibonacci number, and a loop prints the required number of terms.

In [4]:

```
def fib(n):

    if n == 0:
        return 0

    if n == 1:
        return 1

    return fib(n - 1) + fib(n - 2)


terms = 8

print("Fibonacci Series:")

for i in range(terms):
    print(fib(i), end=" ")
```

```
Fibonacci Series:
0 1 1 2 3 5 8 13
```

Question 5 📝 Aim

To calculate the volume of a cone using a lambda function.

Explanation

A lambda function is a short anonymous function used for simple calculations. Here, it calculates the volume of a cone using the formula:

Volume = (1/3) × π × r² × h

In [5]:

```
import math

cone_volume = lambda r, h: (1 / 3) * math.pi * r ** 2 * h

print("Volume =", cone_volume(3, 5))
```

```
Volume = 47.12388980384689
```

Question 6 Aim

To return the maximum and minimum values from a list using a lambda function.

Explanation

The lambda function returns a tuple containing the maximum and minimum values of the list. It uses Python's built-in max() and min() functions for simplicity.

In [6]:
```
data = [10, 6, 8, 90, 12, 56]

max_min = lambda values: (max(values), min(values))

print("Result =", max_min(data))
```

```
Result = (90, 6)
```

Question 7 Aim

To demonstrate keyword arguments, default arguments, and variable-length arguments.

Explanation

Python functions support different types of arguments. Keyword arguments allow values to be passed using parameter names, default arguments provide predefined values when no value is supplied, and variable-length arguments (*args) allow passing any number of values to a function.

In [7]:
```
# Keyword Argument
def student_info(name, age):
    print(f"Name: {name}, Age: {age}")

student_info(age=39, name="Messi")


# Default Argument
def greet(name, message="Welcome to Python Lab"):
    print(message, name)

greet("Rithvik")


# Variable-Length Argument
def add_numbers(*numbers):
    print("Sum =", sum(numbers))

add_numbers(10, 20, 30, 40)
```

```
Name: Messi, Age: 39
Welcome to Python Lab Rithvik
Sum = 100
```

Experiment 8

Experiment 8: File Handling and Exception Handling

Question 1 Aim

To create a text file and write user input into it.

Explanation

The program creates a text file using write (w) mode. It takes input from the user and stores it in the file. If the file already exists, its previous contents are replaced.

In [1]:
```
file = open("student.txt", "w")
text = input("Enter text to write into the file: ")
file.write(text)
file.close()
print("Data written successfully.")
```

```
Data written successfully.
```

Question 2 Aim

To read the contents of a text file.

Explanation

The file is opened in read (r) mode. The read() method retrieves all the data stored in the file and displays it on the screen.

In [2]:
```
file = open("student.txt", "r")
content = file.read()
print(content)
file.close()
```

```
Welcome to Python Lab
```

Question 3 Aim

To append new data to an existing text file.

Explanation

The file is opened in append (a) mode, which adds new data at the end without deleting the existing content.

In [3]:
```
file = open("student.txt", "a")
file.write("\nPython File Handling")
file.close()
```

```
print("Data appended successfully.")
```

```
Data appended successfully.
```

Question 4 Aim

To count the number of lines, words, and characters in a text file.

Explanation

The program reads the complete file and calculates the number of lines, words, and characters using built-in string methods such as split() and len().

In [4]:

```
file = open("student.txt", "r")

content = file.read()

lines = content.split("\n")
words = content.split()

print("Lines =", len(lines))
print("Words =", len(words))
print("Characters =", len(content))

file.close()
```

```
Lines = 2
Words = 7
Characters = 42
```

Question 5 Aim

To handle division by zero using exception handling.

Explanation

The try block contains the code that may generate an error. If the denominator is zero, the except block catches the ZeroDivisionError and displays an appropriate message instead of terminating the program.

In [5]:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))

    result = a / b

    print("Result =", result)

except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

```
Division by zero is not allowed.
```

Question 6 Aim

To handle invalid user input using exception handling.

Explanation

If the user enters a value other than an integer, Python raises a ValueError. The program catches this exception and displays a user-friendly error message.

In [6]:
```
try:
```



```
    print("Please enter a valid integer.")
```

```
Please enter a valid integer.
```

Question 7 Aim

To handle multiple exceptions in a single program.

Explanation

A single try block can have multiple except blocks to handle different types of errors. This makes the program more reliable and prevents unexpected crashes.

In [7]:
```
try:
    num = int(input("Enter a number: "))
    result = 100 / num

    print("Result =", result)

except ZeroDivisionError:
    print("Cannot divide by zero.")

except ValueError:
    print("Invalid input. Please enter a number.")
```

```
Invalid input. Please enter a number.
```

Question 8 Aim

To demonstrate the use of the finally block.

Explanation

The finally block always executes whether an exception occurs or not. It is commonly used to close files, release resources, or display completion messages.

In [8]:
```
try:
    file = open("student.txt", "r")
    print(file.read())

except FileNotFoundError:
    print("File not found.")

finally:
    print("Program execution completed.")
```

```
Welcome to Python Lab
Python File Handling
```

```
Program execution completed.
```

Experiment 9

Experiment 9: Classes and Objects

Question 1 📝 Aim

To create a Student class with student details and display the information of three students.

Explanation

A class is a blueprint used to create objects. In this program, the Student class stores the student's name, SAP ID, and marks. Three objects are created, and their details are displayed using a class method.

In [2]:
```
class Student:

    def __init__(self, name, sap_id, marks):
        self.name = name
        self.sap_id = sap_id
        self.marks = marks

    def display(self):
        print("Name :", self.name)
        print("SAP ID :", self.sap_id)
        print("Marks :", self.marks)
        print()


student1 = Student("KS RITHVIK", "590030656", {"Physics": 88, "Chemistry":
student2 = Student("Rahul", "500082001", {"Physics": 82, "Chemistry": 85, '
student3 = Student("Priya", "500082002", {"Physics": 90, "Chemistry": 92, '

student1.display()
student2.display()
student3.display()
```

```
Name : KS RITHVIK
SAP ID : 590030656
Marks : {'Physics': 88, 'Chemistry': 91, 'Maths': 95}

Name : Rahul
SAP ID : 500082001
Marks : {'Physics': 82, 'Chemistry': 85, 'Maths': 80}

Name : Priya
SAP ID : 500082002
Marks : {'Physics': 90, 'Chemistry': 92, 'Maths': 89}
```

Question 2 📝 Aim

To initialize student details using a constructor and calculate the total marks.

Explanation

The constructor (**init**) automatically initializes object data when an object is created. Separate methods are used to display student information and calculate the total marks, making the program more organized.

In [4]:
```
class StudentRecord:

    def __init__(self, name, sap_id, marks):
        self.name = name
        self.sap_id = sap_id
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("SAP ID:", self.sap_id)
        print("Marks:", self.marks)

    def total_marks(self):
        return sum(self.marks.values())


student = StudentRecord(
    "KS RITHVIK",
    "590030656",
    {"Physics": 90, "Chemistry": 85, "Maths": 95}
)

student.display()
print("Total Marks =", student.total_marks())
```

```
Name: KS RITHVIK
SAP ID: 590030656
Marks: {'Physics': 90, 'Chemistry': 85, 'Maths': 95}
Total Marks = 270
```

Question 3 📝 Aim

To implement different types of inheritance.

Explanation

Inheritance allows one class to use the properties and methods of another class. It promotes code reusability and reduces duplication. This program demonstrates single inheritance.

In [5]:
```
class Person:

    def introduce(self):
        print("I am a person.")


class Teacher(Person):

    def teach(self):
        print("I teach Python.")


teacher = Teacher()
```

```
teacher.introduce()
teacher.teach()
```

```
I am a person.
I teach Python.
```

Question 4 📝 Aim

To demonstrate method overriding.

Explanation

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class. This allows the child class to customize inherited behavior.

In [6]:

```
class Shape:

    def describe(self):
        print("This is a generic shape.")

    def area(self):
        return 0


class Circle(Shape):

    def area(self):
        radius = 5
        return 3.14 * radius * radius


shape = Shape()
circle = Circle()

shape.describe()
print("Shape Area =", shape.area())

circle.describe()
print("Circle Area =", circle.area())
```

```
This is a generic shape.
Shape Area = 0
This is a generic shape.
Circle Area = 78.5
```

Question 5 📝 Aim

To implement operator overloading by adding two Point objects.

Explanation

Operator overloading allows operators such as + to work with user-defined objects. Here, the **add**() method is overloaded so that two Point objects can be added together.

In [7]:

```
class Point:

    def __init__(self, x, y):
        self.x = x
```

```
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def display(self):
        print(f"({self.x}, {self.y})")


p1 = Point(2, 3)
p2 = Point(4, 5)

result = p1 + p2

print("Point 1:", end=" ")
p1.display()

print("Point 2:", end=" ")
p2.display()

print("Sum:", end=" ")
result.display()
```

```
Point 1: (2, 3)
Point 2: (4, 5)
Sum: (6, 8)
```

Experiment 10

Experiment 10: Data Analysis and Visualization

Question 1 📝 Aim

To create a NumPy array and find the sum of all its elements.

Explanation

NumPy is a powerful library used for numerical computations in Python. In this program, a NumPy array is created, and the np.sum() function is used to calculate the sum of all the elements efficiently.

In [1]:

```
import numpy as np

arr = np.array([12, 25, 7, 30, 16])

print("Array:", arr)
print("Sum of all elements:", np.sum(arr))
```

```
Array: [12 25  7 30 16]
Sum of all elements: 90
```

Question 2 📝 Aim

To create a 3 × 3 NumPy array and find the row sums, column sums, and second maximum element.

Explanation

The program demonstrates common NumPy operations. It calculates the sum of each row and column using the axis parameter and finds the second largest element by sorting the array values.

In [2]:
```
import numpy as np

matrix_3x3 = np.array([
    [12, 5, 8],
    [7, 25, 14],
    [3, 19, 11],
])

row_sums = np.sum(matrix_3x3, axis=1)
column_sums = np.sum(matrix_3x3, axis=0)

second_max = np.sort(matrix_3x3, axis=None)[-2]

print("Matrix:\n", matrix_3x3)
print("Row sums:", row_sums)
print("Column sums:", column_sums)
print("Second maximum element:", second_max)
```

```
Matrix:
 [[12  5  8]
 [ 7 25 14]
 [ 3 19 11]]
Row sums: [25 46 33]
Column sums: [22 49 33]
Second maximum element: 19
```

Question 3 📝 Aim

To perform matrix multiplication using NumPy.

Explanation

Matrix multiplication is a common mathematical operation used in machine learning and data analysis. NumPy performs matrix multiplication efficiently using the @ operator.

In [3]:
```
import numpy as np

matrix_a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
])
```